

Python 3 – Befehlsreferenz

Nachschlagewerk für Fachinformatiker · sortiert von einfach nach komplex

Alle Befehle, Funktionen und Syntax-Bausteine aus den Unterrichtsfolien „Python 3 (Fachinformatiker)“, geordnet nach Themen. Jede Zeile: **Befehl/Syntax** · **Bedeutung** · **Beispiel**. Code ist in nichtproportionaler Schrift gesetzt; -> bedeutet „ergibt/liefert“.

1. Variablen & Datentypen

Befehl / Syntax	Bedeutung	Beispiel
<code>x = wert</code>	Zuweisung; Python erkennt den Typ automatisch nach der Zuweisung	<code>s = "Hallo"</code> <code>i = 42</code>
<code>str</code>	Text / Zeichenkette (beliebig lang)	<code>s = "Hallo Welt"</code>
<code>int</code>	Ganzzahl (in Python beliebig groß)	<code>i = 756700919920</code>
<code>float</code>	Fließkommazahl	<code>pi = 3.14159</code>
<code>bool</code>	Wahrheitswert True / False (Großschreibung!)	<code>a = True</code>
<code>list</code>	Liste: mehrere Werte, auch gemischte Typen	<code>l = [1010, "Hose", 9.98, True]</code>
<code>type(x)</code>	Liefert den Datentyp einer Variablen	<code>type(3.14) -> float</code>

2. Casting (Typumwandlung)

Befehl / Syntax	Bedeutung	Beispiel
<code>int(x)</code>	String / Float -> Ganzzahl	<code>int("42") -> 42</code>
<code>float(x)</code>	String / Int -> Fließkommazahl	<code>float("3.14") -> 3.14</code>
<code>str(x)</code>	Zahl -> String (Text)	<code>str(2024) -> '2024'</code>
<code>bool(x)</code>	0 / 1 -> False / True	<code>bool(0) -> False</code>
<code>bin(i)</code>	Integer -> Binärstring	<code>bin(10) -> '0b1010'</code>
<code>list(s)</code>	String / Bereich -> Liste	<code>list('abc') -> ['a', 'b', 'c']</code>
<code>tuple(l)</code>	Liste -> Tupel	<code>tuple([1,2]) -> (1, 2)</code>
<code>set(l)</code>	Liste -> Set (entfernt Duplikate)	<code>set([1,1,2]) -> {1, 2}</code>
<code>round(x, n)</code>	Rundet auf n Nachkommastellen	<code>round(2/3, 2) -> 0.67</code>

3. Ein- & Ausgabe

Befehl / Syntax	Bedeutung	Beispiel
<code>print(a, b, ...)</code>	Ausgabe; Kommas trennen mit Leerzeichen	<code>print("x =", 5)</code>
<code>print(..., end="")</code>	Unterdrückt den Zeilenumbruch am Ende	<code>print("kein Umbruch", end="")</code>
<code>"\n"</code>	Zeilenumbruch innerhalb eines Strings	<code>print("Zeile1\nZeile2")</code>
<code>"a" + "b"</code>	Strings mit + aneinanderhängen	<code>print("Micro" + "soft")</code>
<code>input(text)</code>	Eingabe vom Benutzer (immer als String!)	<code>name = input("Name> ")</code>
<code>int(input(...))</code>	Eingabe direkt in eine Zahl umwandeln	<code>i = int(input("Zahl> "))</code>

4. Arithmetische Operatoren

Befehl / Syntax	Bedeutung	Beispiel
<code>+</code> <code>-</code>	Addition, Subtraktion	<code>-3 - 5 + 10 -> 2</code>
<code>*</code> <code>/</code>	Multiplikation, Division	<code>5 * 2 / 4 -> 2.5</code>
<code>**</code>	Potenzieren	<code>10 ** 3 -> 1000</code>
<code>%</code>	Modulo = Rest einer Division	<code>20 % 6 -> 2</code>

Befehl / Syntax	Bedeutung	Beispiel
//	Ganzzahldivision (abrunden)	29 // 10 -> 2
+= -=	Wert erhöhen / vermindern (Inkrement)	i += 1
+ (Text)	Strings aneinanderhängen	"Apollo " + "11"
* (Text)	String vervielfachen	16 * "+" -> "++++..."

5. Vergleichs- & logische Operatoren

Befehl / Syntax	Bedeutung	Beispiel
== !=	Gleich / ungleich	3 == 3 -> True
< > <= >=	Kleiner / größer (gleich)	5 <= 5 -> True
and or not	Logische Verknüpfung von Bedingungen	(3>2) and not(4>3) -> False
in	Enthalten in Liste / Set / String	3 in [1,2,3] -> True

6. Bitweise Operatoren

Befehl / Syntax	Bedeutung	Beispiel
& ^	Bitweises OR, AND, XOR (Ganzzahlen)	0b1001 0b0110 -> 15
<< >>	Bits nach links / rechts schieben	0b001 << 2 -> 0b100
bin(i)	Dezimalzahl -> Dualdarstellung	bin(10) -> '0b1010'

7. Verzweigungen (if)

Befehl / Syntax	Bedeutung	Beispiel
if bed:	Block ausführen, wenn Bedingung True ist	if x < 0:
elif bed:	Weitere Prüfung (else if)	elif x > 200:
else:	Sonst-Zweig (wenn keine Bedingung zutraf)	else:
if a and (b or c):	Bedingungen mit and/or/not kombinieren	if a>3 and (c!=22 or d==1):
match / case	Mehrfachverzweigung (ab Python 3.10)	match wert: case 1: ...

8. Schleifen

Befehl / Syntax	Bedeutung	Beispiel
for i in range(n):	Zählschleife, i läuft von 0 bis n-1	for i in range(10):
range(a, b)	Von a bis b-1	range(5, 20)
range(a, b, s)	Mit Schrittweite s	range(0, 100, 10)
range(a, b, -1)	Rückwärts zählen	range(10, -1, -1)
while bed:	Kopfgesteuert: läuft, solange Bedingung True	while i < 10:
while True:	Endlosschleife (Abbruch nur per break)	while True:
break	Schleife sofort verlassen	if x == 0: break
continue	Rest des Durchlaufs überspringen	if x < 0: continue
for x in liste:	Direkt über die Elemente iterieren	for auto in autos:

9. Listen - Grundlagen & Zugriff

Befehl / Syntax	Bedeutung	Beispiel
l = [a, b, c]	Liste erzeugen	autos = ["Vw", 35000, True]
l[0]	Erstes Element (Index ab 0)	autos[0] -> "Vw"
l[-1]	Letztes Element	autos[-1]
l[0:3]	Teilbereich (Slicing): Element 0 bis 2	l[0:3]

Befehl / Syntax	Bedeutung	Beispiel
<code>l[:2] l[-3:]</code>	Erste 2 / letzte 3 Elemente	<code>l[:2]</code>
<code>l[i] = wert</code>	Einzelnes Element ändern	<code>autos[0] = "Opel"</code>
<code>len(l)</code>	Anzahl der Elemente	<code>len(autos)</code>

10. Listen - Methoden & Funktionen

Befehl / Syntax	Bedeutung	Beispiel
<code>l.append(x)</code>	Ein Element ans Ende anhängen	<code>autos.append("ID.3")</code>
<code>l.extend([.])</code>	Mehrere Elemente anhängen	<code>l.extend(["a", "b"])</code>
<code>l.insert(i, x)</code>	Element an Position i einfügen	<code>l.insert(0, "Trabant")</code>
<code>del l[i]</code>	Element an Position i löschen	<code>del l[0]</code>
<code>l.pop()</code>	Letztes Element entfernen (und liefern)	<code>l.pop()</code>
<code>l.remove(x)</code>	Erstes Element mit Inhalt x löschen	<code>l.remove("Meier")</code>
<code>l.index(x)</code>	Position von x in der Liste finden	<code>l.index("Meier")</code>
<code>l.sort()</code>	Liste sortieren (verändert das Original!)	<code>l.sort(reverse=True)</code>
<code>sorted(l)</code>	Sortiert zurückgeben (Original bleibt)	<code>sorted(l)</code>
<code>l.copy()</code>	Liste kopieren (duplizieren)	<code>b = a.copy()</code>
<code>min / max / sum(l)</code>	Kleinstes / größtes Element, Summe	<code>sum([1,2,3]) -> 6</code>
<code>list(range(a,b,s))</code>	Liste aus einem Zahlenbereich erzeugen	<code>list(range(1, 50, 2))</code>
<code>[0] * n</code>	Liste mit n gleichen Werten	<code>[0] * 10000</code>
<code>l.sort(key=lambda x: x[2])</code>	Nach Element einer Unterliste sortieren	<code>kunden.sort(key=lambda x: x[2])</code>

11. Tupel, Sets & Dictionaries

Befehl / Syntax	Bedeutung	Beispiel
<code>t = (a, b)</code>	Tupel: wie Liste, aber unveränderbar	<code>t = ("Fritz", 350, True)</code>
<code>s = {a, b, c}</code>	Set: jeder Wert kommt nur einmal vor	<code>s = {7, 5, 3, 6}</code>
<code>s.add(x) / s.discard(x)</code>	Element hinzufügen / entfernen	<code>s.add(9)</code>
<code>d = {key: value}</code>	Dictionary: Schlüssel-Wert-Paare	<code>d = {"temp": 22.5}</code>
<code>d["key"]</code>	Zugriff über den Schlüssel	<code>d["temp"] -> 22.5</code>
<code>d["neu"] = v</code>	Paar hinzufügen oder ändern	<code>d["land"] = "DE"</code>
<code>.items() .keys() .values()</code>	Paare / Schlüssel / Werte (für Schleifen)	<code>for k, v in d.items():</code>
<code>d.get("key")</code>	Wert holen, ohne Fehler bei fehlendem Key	<code>d.get("x") -> None</code>
<code>del d["key"]</code>	Ein Paar löschen	<code>del d["temp"]</code>
<code>len(d)</code>	Anzahl der Paare	<code>len(d)</code>

12. Strings (Methoden & Funktionen)

Befehl / Syntax	Bedeutung	Beispiel
<code>s[0:5]</code>	Die ersten 5 Zeichen	<code>name[0:5]</code>
<code>s[-3:]</code>	Ab dem drittletzten Zeichen	<code>name[-3:]</code>
<code>s.upper() / s.lower()</code>	In Groß- / Kleinbuchstaben	<code>"abc".upper() -> "ABC"</code>
<code>s.strip()</code>	Leerzeichen am Anfang/Ende entfernen	<code>" hi ".strip() -> "hi"</code>
<code>s.replace(a, b)</code>	Teilstring a durch b ersetzen	<code>s.replace("a", "b")</code>
<code>s.split(t)</code>	An t aufteilen -> Liste von Teilstrings	<code>"a;b".split(";") -> ["a","b"]</code>
<code>s.find(t)</code>	Index der ersten Fundstelle	<code>"abc".find("b") -> 1</code>
<code>s.count(t)</code>	Anzahl der Vorkommen von t	<code>"aaa".count("a") -> 3</code>

Befehl / Syntax	Bedeutung	Beispiel
<code>len(s)</code>	Anzahl der Zeichen (Länge)	<code>len("Hallo") -> 5</code>
<code>ord(c) / chr(i)</code>	Zeichen <-> Unicode-Nummer	<code>ord("A") -> 65 ; chr(65) -> "A"</code>

13. f-Strings & Formatierung

Befehl / Syntax	Bedeutung	Beispiel
<code>f"{var}"</code>	Variable direkt in den String einsetzen	<code>f"Hallo {name}"</code>
<code>{var:25}</code>	Feldbreite: 25 Stellen	<code>f"{name:25}"</code>
<code>{stk:4d}</code>	Ganzzahl, 4 Stellen breit	<code>f"{stk:4d}"</code>
<code>{plz:05d}</code>	5 Stellen mit führenden Nullen	<code>f"{plz:05d}" -> "00531"</code>
<code>{preis:8.2f}</code>	8 Stellen, 2 Nachkommastellen	<code>f"{preis:8.2f} EUR"</code>
<code>{nr:<10d} / {nr:>10d}</code>	Links- / rechtsbündig	<code>f"{nr:<10d}"</code>
<code>{ort:^16s}</code>	Zentriert, 16 Stellen breit	<code>f"{ort:^16s}"</code>
<code>{masse:12,.3f}</code>	1000er-Trennung, 3 Nachkommastellen	<code>f"{masse:12,.3f}"</code>
<code>{rabatt:.2%}</code>	Prozent (x100), 2 Nachkommastellen	<code>f"{rabatt:.2%}" -> "15.00%"</code>
<code>{addr:x} / {addr:b}</code>	Als Hexadezimal / Binärzahl	<code>f"{addr:x}"</code>

14. Funktionen & Prozeduren

Befehl / Syntax	Bedeutung	Beispiel
<code>def name(args):</code>	Unterprogramm definieren	<code>def vol(b, h, t):</code>
<code>return wert</code>	Wert zurückgeben -> es ist eine Funktion	<code>return b * h * t</code>
<code>def ... (ohne return)</code>	Prozedur: gibt etwas aus, liefert nichts zurück	<code>def printvol(b,h,t): print(b*h*t)</code>
<code>name(args)</code>	Funktion / Prozedur aufrufen	<code>y = vol(5, 10, 25)</code>
<code>if __name__ == "__main__":</code>	Code nur bei direktem Start ausführen (nicht bei import)	<code>if __name__ == "__main__":</code>

15. Module & import

Befehl / Syntax	Bedeutung	Beispiel
<code>import modul</code>	Ganzes Modul; Aufruf mit modul.funktion()	<code>import math</code>
<code>from modul import f</code>	Nur eine Funktion importieren	<code>from math import pi</code>
<code>from modul import *</code>	Alle Funktionen (ohne Modulname nutzbar)	<code>from math import *</code>
<code>import modul as x</code>	Modul unter einem Kurznamen importieren	<code>import random as r</code>
<code>dir(modul)</code>	Zeigt die Funktionen eines Moduls	<code>dir(random)</code>
<code>help(f)</code>	Zeigt die Hilfe zu Funktion / Modul	<code>help(random.randint)</code>
<code>pip3 install modul</code>	Modul installieren (im Terminal/cmd)	<code>pip3 install requests</code>

16. Modul math

Befehl / Syntax	Bedeutung	Beispiel
<code>math.sqrt(n)</code>	Quadratwurzel von n	<code>math.sqrt(9) -> 3.0</code>
<code>math.ceil(f) / floor(f)</code>	Auf- / abrunden	<code>math.ceil(2.1) -> 3</code>
<code>math.pi</code>	Die Kreiszahl Pi	<code>math.pi -> 3.14159...</code>
<code>math.sin/cos/tan(x)</code>	Trigonometrie (Radian-Werte)	<code>math.sin(0) -> 0.0</code>
<code>math.log(x, b)</code>	Logarithmus von x zur Basis b	<code>math.log(8, 2) -> 3.0</code>
<code>math.gcd(a, b)</code>	Größter gemeinsamer Teiler	<code>math.gcd(12, 8) -> 4</code>
<code>math.factorial(n)</code>	Fakultät n!	<code>math.factorial(5) -> 120</code>

17. Modul random (Zufallszahlen)

Befehl / Syntax	Bedeutung	Beispiel
<code>random.seed(n)</code>	Startwert setzen (reproduzierbar)	<code>random.seed(1)</code>
<code>random.random()</code>	Float-Zufallszahl: $0 \leq x < 1$	<code>random.random()</code>
<code>random.uniform(a, b)</code>	Float-Zufallszahl zwischen a und b	<code>random.uniform(0, 1)</code>
<code>random.randint(a, b)</code>	Ganzzahl a bis b (b inklusive!)	<code>random.randint(0, 99)</code>
<code>random.randrange(a, b)</code>	Ganzzahl a bis b-1	<code>random.randrange(0, 99)</code>
<code>random.choice(l)</code>	Zufälliges Element aus einer Liste	<code>random.choice(["Ja", "Nein"])</code>
<code>random.shuffle(l)</code>	Liste zufällig durchmischen	<code>random.shuffle(liste)</code>

18. Module time & datetime

Befehl / Syntax	Bedeutung	Beispiel
<code>import time</code>	Zeit-Modul einbinden	<code>import time</code>
<code>time.time()</code>	Sekunden seit 1.1.1970 (Epoch)	<code>t = time.time()</code>
<code>time.sleep(s)</code>	Programm s Sekunden pausieren	<code>time.sleep(2.5)</code>
<code>time.ctime(t)</code>	Epoch -> lesbarer String	<code>time.ctime(t)</code>
<code>time.localtime()</code>	struct_time der aktuellen Zeit	<code>time.localtime()</code>
<code>import datetime</code>	Datum/Uhrzeit-Modul einbinden	<code>import datetime</code>
<code>datetime.datetime.now()</code>	Aktuelles Datum + Uhrzeit	<code>jetzt = datetime.datetime.now()</code>
<code>obj.strftime(fmt)</code>	datetime -> formatierter String	<code>jetzt.strftime("%d.%m.%Y")</code>
<code>datetime.strptime(s, fmt)</code>	String -> datetime-Objekt	<code>strptime("08.11.1895", "%d.%m.%Y")</code>

19. Fehler abfangen (try / except)

Befehl / Syntax	Bedeutung	Beispiel
<code>try:</code>	Code, der einen Laufzeitfehler auslösen kann	<code>try: print(1/i)</code>
<code>except:</code>	Fehlerbehandlung (fängt alle Fehler)	<code>except: print("Fehler!")</code>
<code>except ZeroDivisionError:</code>	Einen bestimmten Fehler abfangen	<code>except ZeroDivisionError:</code>
<code>except Exception as e:</code>	Fehlerobjekt selbst anzeigen	<code>except Exception as e: print(e)</code>

20. Dateien (Text)

Befehl / Syntax	Bedeutung	Beispiel
<code>open(name, modus)</code>	Datei öffnen -> Dateiobjekt	<code>f = open("d.txt", "w")</code>
<code>"w" / "r" / "a"</code>	Schreiben (überschreibt) / Lesen / Anhängen	<code>open("d.txt", "a")</code>
<code>"rb" / "wb"</code>	Binärmodus lesen / schreiben	<code>open("d.pkl", "wb")</code>
<code>encoding="utf-8"</code>	Zeichencodierung festlegen	<code>open(name, "w", encoding="utf-8")</code>
<code>f.write(x)</code>	String in die Datei schreiben	<code>f.write("Zeile\n")</code>
<code>f.read(n)</code>	n Bytes lesen, oder ganze Datei	<code>f.read()</code>
<code>f.readline() / readlines()</code>	Eine Zeile / alle Zeilen (als Liste)	<code>f.readlines()</code>
<code>for z in f:</code>	Datei zeilenweise durchlaufen	<code>for zeile in f: print(zeile)</code>
<code>f.close()</code>	Datei schließen (beim Schreiben wichtig!)	<code>f.close()</code>
<code>with open(...) as f:</code>	Datei automatisch schließen (kein close nötig)	<code>with open("d.txt", "r") as f:</code>

21. CSV-Dateien

Befehl / Syntax	Bedeutung	Beispiel
<code>import csv</code>	CSV-Modul einbinden	<code>import csv</code>
<code>csv.reader(fo)</code>	Lese-Objekt erzeugen	<code>r = csv.reader(fo, delimiter=";")</code>
<code>for zeile in reader:</code>	CSV zeilenweise lesen (jede Zeile = Liste)	<code>for zeile in r:</code>
<code>next(reader)</code>	Eine Zeile überspringen (z.B. Überschrift)	<code>ueberschrift = next(r)</code>
<code>csv.writer(fo)</code>	Schreib-Objekt erzeugen	<code>w = csv.writer(fo, delimiter=";")</code>
<code>w.writerow(liste)</code>	Eine Zeile schreiben	<code>w.writerow(["Name", "Anzahl"])</code>
<code>w.writerows(liste2d)</code>	Mehrere Zeilen (2D-Liste) schreiben	<code>w.writerows(daten)</code>
<code>delimiter=";"</code>	Trennzeichen festlegen (oft ; statt ,)	<code>delimiter=";"</code>

22. JSON

Befehl / Syntax	Bedeutung	Beispiel
<code>import json</code>	JSON-Modul einbinden	<code>import json</code>
<code>json.loads(s)</code>	JSON-String -> Dictionary	<code>d = json.loads(text)</code>
<code>json.dumps(d)</code>	Dictionary -> JSON-String	<code>json.dumps(d, indent=4)</code>
<code>json.load(fo)</code>	JSON-Datei -> Dictionary	<code>d = json.load(fo)</code>
<code>json.dump(d, fo)</code>	Dictionary -> JSON-Datei	<code>json.dump(d, fo, indent=4)</code>

23. Datenbank (sqlite3)

Befehl / Syntax	Bedeutung	Beispiel
<code>import sqlite3</code>	DB-Modul (in Python enthalten)	<code>import sqlite3</code>
<code>sqlite3.connect(datei)</code>	Verbindung herstellen (legt DB ggf. an)	<code>db = sqlite3.connect("db.sqlite")</code>
<code>db.cursor()</code>	Cursor zum Ausführen von SQL erzeugen	<code>cur = db.cursor()</code>
<code>cur.execute(sql)</code>	SQL-Befehl ausführen	<code>cur.execute("SELECT * FROM t")</code>
<code>cur.execute(sql, [a, b])</code>	Mit Parameterbindung (? im SQL-String)	<code>cur.execute(sql, [name, anz])</code>
<code>db.commit()</code>	Änderungen speichern (INSERT/UPDATE/DELETE!)	<code>db.commit()</code>
<code>cur.fetchall()</code>	Alle Ergebniszeilen als Liste holen	<code>data = cur.fetchall()</code>
<code>cur.fetchone()</code>	Eine Ergebniszeile holen	<code>ds = cur.fetchone()</code>
<code>for ds in cur:</code>	Ergebnis durchlaufen	<code>for ds in cur: print(ds[0])</code>
<code>db.close()</code>	Verbindung schließen	<code>db.close()</code>

24. Wichtige SQL-Befehle (mit cursor.execute)

Befehl / Syntax	Bedeutung	Beispiel
<code>SELECT ... FROM ...</code>	Daten aus der Datenbank auslesen	<code>SELECT * FROM tblArtikel</code>
<code>INSERT INTO ... VALUES ...</code>	Datensatz einfügen	<code>INSERT INTO tbl(Name) VALUES('Hut')</code>
<code>UPDATE ... SET ... WHERE ...</code>	Daten ändern	<code>UPDATE tbl SET Anzahl=5 WHERE id=1</code>
<code>DELETE FROM ... WHERE ...</code>	Datensatz löschen	<code>DELETE FROM tbl WHERE id=1</code>
<code>CREATE TABLE ...</code>	Tabelle anlegen	<code>CREATE TABLE tbl(id INTEGER, Name TEXT)</code>
<code>ALTER TABLE ...</code>	Tabellenaufbau ändern	<code>ALTER TABLE tbl ADD COLUMN Preis REAL</code>
<code>DROP TABLE ...</code>	Tabelle löschen	<code>DROP TABLE tbl</code>

25. Weitere Module (Überblick)

Befehl / Syntax	Bedeutung	Beispiel
<code>pickle</code>	Objekte binär speichern / laden	<code>pickle.dump(obj, fo) / pickle.load(fo)</code>
<code>pandas (as pd)</code>	Tabellen analysieren (DataFrame)	<code>pd.read_csv("d.csv", delimiter=";")</code>
<code>requests</code>	Web-APIs (REST) abfragen	<code>requests.get(url).json()</code>
<code>matplotlib.pyplot (as plt)</code>	Diagramme zeichnen	<code>plt.bar(x, y); plt.show()</code>
<code>xml.etree.ElementTree (as ET)</code>	XML-Dateien lesen / schreiben	<code>ET.parse("d.xml").getroot()</code>
<code>locale</code>	Länderspezifische Zahlen/Datumsformate	<code>locale.setlocale(locale.LC_ALL, 'de_de')</code>